Michael Warner

Sudoku ReAct Agent

Git repo: https://github.com/michaelwarner27/AI4300/tree/main/U0-HW-08
Commit Hash:
https://github.com/michaelwarner27/AI4300/commit/5dfda301f5d9ace7872651205357ea02d0d6ccb1

ReAct Loop
Prompt Message
↓
Initialize/Update Knowledge Base (before LLM decision)
↓
LLM Decision/Response (only node making API calls)
↓
Response Parser (validate tool call format)
↓
Execute Tool Call
↓
Tool Result + validateIncompleteBoard() (only after writes)
↓
Return to LLM Decision → Loop

Development Process:

I began this process by writing out a requirements document for my Sudoku board. I then asked my LLM what was missing or unclear from my documentation, and iterated through that until I was happy. I then asked it to reformat my requirements as an easy to read .md file. You can see these in ./Human-Requirements/board-requirements.txt and ./Agent-Requirements/board-req.md. I then had my LLM build Sudoku based on these requirements.

I then followed the same process for the ReAct agent although this one took a lot more iteration to get it working. It was difficult to figure out what solving strategy would work for the agent, and I still didn't get it good.

Testing:

The tests were written by a new session of my LLM based upon the documentation written. It was honestly annoying to keep updating the documentation every time I made a change to the program so I think next time I will add a clause in and agents.md to have it write changes to the documentation.

What I would Improve:

In the current version of my agent I leave too much choice up to it, so it usually gets stuck in loops of reading the board over and over, sometimes adding clauses to the KB, but it didn't really know when to solve. I would use the deterministic side to suggest when to check

tiles. For example I would start by feeding the agent all the tiles that only have 1 legal option so that it doesn't have to read the notes on every tile to fill those in. I would also be more specific in my prompting on when to read and add clause vs when to check for entailment.

I would also add a tool that allows the agent to read the knowledge base, because of the limited context window it doesn't know how many clauses it has written so it doesn't know how to solve. I was actually surprised by how many times I needed to change my tools because my original ones were useless to my agent.

Reflection on AI Assisted Development:

This was the first assignment that I actually felt good about AI development. I was significantly more in control and understood what my agent was doing. Right now I am paying for tokens so I was more conscientious with my context windows, and learned how important it is for me to be familiar with the entire project to direct my agent exactly to where I want it. An example of this was my agent created a second function to render the sudoku board to the terminal because the existing one was out of its context, and I was able to catch that and say no this function has already been implemented, use the existing one.

Prompting the ReAct agent to solve the board the way I wanted was really difficult and I didn't do it successfully. Sometimes it would ignore the tool calls and try to give me a solved board. Other times it got stuck reading the board forever and never checking for entailment, or writing to cells.

Overall I feel like I've finally made progress in understanding how to develop this way. I was significantly less stuck on the low level writing of functions and entirely focused on the higher level logic. Up to this point I've honestly hated all AI development but I think I can finally see that when done properly, it is extremely valuable.

Final Takeaway:

I definitely need more practice, but focusing on building requirement docs first significantly helped me understand the project and stay in control. I think I need to make it into more of a loop where I write docs -> implement -> test then back to writing the docs. This time around I skipped from testing back to implementation.